COMP1036 Coursework 2 (Part 1) (15 marks)

Release Date: 28 November 2025 17:00

Deadline: 19 December 2025 17:00

[Note: Coursework 2 (Part 2) (10 marks) – 5 Weekly Quizzes]

Tasks

Write a program in Hack Assembly Language that sorts an array of integers in ascending or descending order. The unsorted array contains 5 to 9 elements, located at a range of memory locations starting from RAM[50]. The integers in the array range from -9 to 9, inclusive.

The program should allow you to sort either the entire array or a portion of it. The number of elements to be sorted is determined by the integers stored in RAM[0] and RAM[1], as described below:

Read two input values, X and Y, from RAM[0] and RAM[1], respectively, and output the computed result, Z, to RAM[2]. X and Y must be non-zero integers (either positive or negative) in the range from -9 to 9, inclusive. The program should function correctly regardless of whether $X < Y$, $X > Y$, or $X = Y$.

Different rules will be applied based on whether X and Y are even or odd integers, as follows:

(1) IF both X and Y are even integers, THEN Z is the maximum number minus the minimum number in the array.
(2) IF both X and Y are odd integers, THEN Z is the minimum number minus the maximum number in the array.
(3) IF one of X or Y is odd and the other is even, THEN Z is the sum of all integers between X and Y (inclusive).
(4) IF X = Y, THEN Z = X or Z = Y.
(5) IF Z is positive, THEN sort the array in ascending order.
(6) IF Z is negative, THEN sort the array in descending order.
(7) IF Z is zero, THEN no sorting should be done.

Example 1:
Given RAM[0] = X = -4; RAM[1] = Y = 2
Maximum number in the array = 5
Minimum number in the array = 1
Applying Rule (1): RAM[2] = Z = 5 – 1 = 4
Applying Rule (5) for Z = 4: Sort the first 4 elements of the array in ascending order.

Example 2:
Given RAM[0] = X = -3; RAM[1] = Y = -1
Maximum number in the array = 5
Minimum number in the array = 1
Applying Rule (2): RAM[2] = Z = 1-5 = -4
Applying Rule (7) for Z = -4: Sort the first 4 elements of the array in descending order

Example 3:

Given RAM[0] = X = -4; RAM[1] = Y = 5

The range of integers between X and Y (inclusive) is [-4, -3, -2, -1, 0, 1, 2, 3, 4, 5].

Applying Rule (3): RAM[2] = Z = (-4)+(-3)+(-2)+(-1)+0+1+2+3+4+5 = 5

Applying Rule (5) for Z = 5: Sort the first 5 elements of the array in ascending order.

Example 4:

Given RAM[0] = X = 3; RAM[1] = Y = 3

Applying Rule (4): RAM[2] = Z = 3

Applying Rule (5) for Z = 3: Sort the first 3 elements of the array in ascending order.

Some Input and Output Examples:

	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out
RAM[0]	1		-2		2		-4		-3		-4		-5		-5		3		5	
RAM[1]	1		3		3		2		-1		5		4		5		3		-5	
RAM[2]		1		3		5		4		-4		5		-5		-4		3		-4
Ignore the values in RAM[3], RAM[4], …, RAM[49]																				
RAM[50]	3	3	3	2	3	1	3	1	3	5	-3	-5	-3	-1	3	5	3	2	-3	-1
RAM[51]	2	2	2	3	2	2	2	2	2	3	-2	-4	-2	-2	2	3	2	3	-2	-2
RAM[52]	5	5	5	5	5	3	5	3	5	2	-5	-3	-5	-3	5	2	5	5	-5	-3
RAM[53]	1	1	1	1	1	4	1	5	1	1	-1	-2	-1	-4	1	1	1	1	-1	-5
RAM[54]	4	4	4	4	4	5	4	4	4	4	-4	-1	-4	-5	4	4	4	4	-4	-4

Requirements

1. RAM[0] and RAM[1] are reserved for holding the two input values, X and Y, respectively.
2. RAM[2] is reserved for the value Z, which is the number of elements in the array to be sorted. This value is computed using the integers stored in RAM[0] and RAM[1], as well as the array's maximum and minimum values.
3. RAM[50] onwards are reserved for holding the array integers before sorting, as well as the results after sorting.
4. Sample test cases are provided to evaluate your program, but the final test cases will be different from the sample test cases.
5. You should use the sample test file to evaluate your program. Your program will be finally evaluated using a similar test file. Failure to comply with the test file format may result in your program failing the final test cases.
6. You should anticipate that the array size in the final test cases may vary, potentially exceeding 5 but not surpassing 9.
7. However, you do not need to be concerned about scenarios where the value Z in RAM[2] exceeds the size of the array in a given test case.
8. Ensure that your program is bug-free. A zero mark will be immediately given to non-executable code.
9. You should complete all tasks in one assembly file. Name your assembly file "**main.asm**".

Marking Criteria

1. 15 marks in total.
2. 15 test cases will be used, each test case carrying one mark.
3. The efficiency of your program will also be considered. A limited number of tick-tocks will be given for each test case. If your program does not generate the expected results within the given number of tick-tocks, you will lose one mark for that test case.

Plagiarism

If you use code from a textbook or the web, you must acknowledge it. Plagiarism detection tools will be used to check for similarities between submissions and web-based material. You are reminded of the school's policy on plagiarism.

How to submit?

You should zip your file "**main.asm**" into one zip file. Name your zip file: YOURSTUDENTID_YOURNAME.zip (e.g., **20514000_Danting_Wang.zip**).

Remember to include your name and student ID at the beginning of the "**main.asm**" file as the first comment line. Submit your zip file to the Moodle page. Please note that each new submission overwrites all the files from the previous one. If you submit several times, ensure that your last submission is the correct version.

Check the submission after you submit it. There have been instances in the past where the submitted file was corrupted. Ensure that the submitted file is complete and executable. You will receive zero marks if your submitted file is corrupted or not executable.

For late submissions, the standard late submission policy applies: a 5% marks deduction for every 24 hours, including weekends and public holidays.